



University of Moratuwa, Sri Lanka

Department of Electronic & Telecommunication Engineering

EN3150 Pattern Recognition

Assignment 02

Learning from Data and Related Challenges
and Classification

Rathnayake R.M.D.D.

220525K

All source code, notebooks and figures for this report are available at
github.com/DDRMin/EN3150_Assignment_02

August 24, 2026

Contents

1	Linear Regression, Outliers, and Regularization	1
1.1	Sensitivity of OLS to Severe Vertical Outliers	1
1.2	Ordinary Least-Squares Fit	1
1.3	Weighted Least Squares	4
1.4	Regularization in High Dimensions	8
1.5	Ridge against Lasso	11
1.6	Necessity of Standardized Feature Scaling	12
2	Logistic Regression: Preprocessing, Interpretation, and Evaluation	13
2.1	String-Valued Columns and Label Encoding	14
2.2	Preprocessing Pipeline and Stratified Split	15
2.3	Grid Search over Regularization Strength and Penalty	16
2.4	Test-Set Evaluation and Class Imbalance	18
2.5	Decision Threshold and the Precision-Recall Trade-off	21
2.6	Direct Probability Computation	23
3	Optimization of Logistic Regression	26
3.1	Gradient and Hessian of the Binary Cross-Entropy Loss	27
3.2	Numerically Stable Sigmoid and Cross-Entropy Loss	28
3.3	Batch Gradient Descent	30
3.4	Mini-Batch Gradient Descent	32
3.5	Damped Newton's Method	34
3.6	Convergence Comparison	35
3.7	Effect of Feature Scaling on Conditioning	37

List of Figures

1	OLS fit on the synthetic data. The dotted vertical segments mark the residual of each injected outlier against the fitted line.	3
2	Weighted against ordinary least squares under known outliers. Down-weighting the twelve contaminated points recovers a line almost coincident with the true one.	8
3	Fitted coefficients on one informative and nine pure-noise features. Lasso eliminates every noise coefficient exactly; Ridge shrinks them but retains all nine. .	10
4	Confusion matrix of the tuned classifier on the test set.	19
5	Precision-recall trade-off on the test set, with the $p_{th} = 0.50$ and $p_{th} = 0.35$ operating points marked.	22
6	Training loss progression of mini-batch gradient descent against batch gradient descent. One batch iteration and one mini-batch epoch each cost a single pass over the training set.	33
7	Convergence of batch gradient descent against damped Newton's method, loss on a logarithmic scale.	36

List of Tables

1	Fitted parameters and mean squared errors of the OLS model.	3
2	OLS against WLS with $a_i = 0.05$ on the known outliers. The inlier-only test MSE excludes the two contaminated test rows.	6
3	Optimal regularization strengths and cross-validated errors on the expanded ten-feature dataset.	9
4	Mean 5-fold cross-validated F_1 -score for every combination in the grid.	17
5	Test-set performance of the tuned logistic regression model.	19
6	Confusion matrix on the 120 test samples.	19
7	Effect of lowering the decision threshold from 0.50 to 0.35.	21
8	Batch gradient descent with $\eta = 0.1$, $T = 300$ and tolerance 10^{-8}	31
9	Mini-batch gradient descent with $B = 32$, $\eta = 0.05$ and $E = 100$ epochs. . . .	34
10	Damped Newton's method against batch gradient descent on the same training set.	35

1. Linear Regression, Outliers, and Regularization

The synthetic dataset used throughout this section is generated from the true line $y = 2.5 + 1.8x$ with additive Gaussian noise $\mathcal{N}(0, 0.8)$, after which twelve of the 160 observations are corrupted by large positive shifts in the range $[10, 15]$. The index number 220525 is used as the random seed.

Listing 1: Generation of the synthetic dataset with twelve injected vertical outliers, seeded with index number 220525.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.model_selection import train_test_split
4 from sklearn.linear_model import LinearRegression, Ridge, Lasso
5 from sklearn.metrics import mean_squared_error
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.pipeline import Pipeline
8
9 indexno = 220525 # index number, used as the fixed random seed
10
11 rng = np.random.default_rng(indexno)
12 X = rng.uniform(-3, 3, size=(160, 1)) # 160 samples, one feature in [-3, 3]
13 y = 2.5 + 1.8 * X[:, 0] + rng.normal(0, 0.8, size=160) # true Line y = 2.5 + 1.8x plus N(0, 0.8) noise
14
15 # Add large errors to 12 observations.
16 outlier_idx = rng.choice(len(y), size=12, replace=False) # rows chosen to be corrupted
17 y[outlier_idx] += rng.uniform(10, 15, size=12) # shift them up by 10-15 (vertical outliers)
```

1.1 Sensitivity of OLS to Severe Vertical Outliers

Explain why Ordinary Least Squares (OLS) estimation is particularly sensitive to severe vertical outliers in y . Reference the squared loss function $J(\mathbf{w}) = \frac{1}{2N} \sum_i (y_i - \mathbf{w}^T \mathbf{x}_i)^2$ in your answer.

OLS gets $\mathbf{w}$ by minimizing $J(\mathbf{w}) = \frac{1}{2N} \sum_i (y_i - \mathbf{w}^T \mathbf{x}_i)^2$. The sensitivity to outliers directly comes from the squares of the residuals.

A residual of an inlier is very small, like $\sigma = 0.7$, so the $\sigma^2 = 0.49$.

But the residual of an outlier is more like 10 - 15 and the squared error is 100 - 255. Therefore vertical outliers contribute strongly to the $J(\mathbf{w})$.

When trying to minimize, OLS pulls the line towards the outliers while inlier residuals grow slightly, distorting the true line.

1.2 Ordinary Least-Squares Fit

Fit an ordinary least-squares (OLS) model to the generated data. Report its fitted intercept (w_0), slope coefficient (w_1), training Mean Squared Error (MSE_{train}), and test Mean

Squared Error (MSE_{test}). Provide a scatter plot showing the data points along with the fitted regression line.

Listing 2: Train/test split, OLS fit, and the scatter plot of the fitted line.

```

1  # 80:20 train/test split (as per the assignment instructions). The row indices
2  # are split alongside the data so Q3 can tell which rows are the outliers.
3  idx_all = np.arange(len(y))
4  X_train, X_test, y_train, y_test, idx_train, idx_test = train_test_split(
5      X, y, idx_all, test_size=0.20, random_state=42)
6
7  ols = LinearRegression()      # OLS on the training split
8  ols.fit(X_train, y_train)
9
10 w0 = ols.intercept_           # fitted intercept
11 w1 = ols.coef_[0]             # fitted slope
12 mse_train = mean_squared_error(y_train, ols.predict(X_train))
13 mse_test = mean_squared_error(y_test, ols.predict(X_test))
14
15 print(f"Intercept (w0) = {w0:.4f}")
16 print(f"Slope (w1) = {w1:.4f}")
17 print(f"MSE train = {mse_train:.4f}")
18 print(f"MSE test = {mse_test:.4f}")
19
20 # Flatten to 1-D for plotting
21 Xf, X_tr, X_te = (np.asarray(a).ravel() for a in (X, X_train, X_test))
22 yf, y_tr, y_te = (np.asarray(a).ravel() for a in (y, y_train, y_test))
23
24 x_line = np.linspace(Xf.min() - 0.3, Xf.max() + 0.3, 200)
25
26 fig, ax = plt.subplots(figsize=(8, 5.5))
27
28 # Candidate Lines
29 ax.plot(x_line, w0 + w1 * x_line, color='#ef6c00', linewidth=2,
30         label=f'OLS fit: $y = {w0:.2f} + {w1:.2f}x$')
31 ax.plot(x_line, 2.5 + 1.8 * x_line, color='#2e7d32', linewidth=2, linestyle='--',
32         label=f'True line: $y = 2.5 + 1.8x$')
33
34 # Data
35 ax.scatter(X_tr, y_tr, s=45, color='#1f3b73', zorder=3,
36            edgecolor='black', linewidth=0.6, label='Training data')
37 ax.scatter(X_te, y_te, s=55, marker='^', color='#00838f', zorder=3,
38            edgecolor='black', linewidth=0.6, label='Test data')
39 ax.scatter(Xf[outlier_idx], yf[outlier_idx], s=170, facecolors='none',
40            edgecolors='#b71c1c', linewidth=1.8, zorder=4,
41            label='Injected outliers')
42
43 # Residual of each outlier against the OLS fit
44 for i in np.atleast_1d(outlier_idx):
45     ax.vlines(Xf[i], w0 + w1 * Xf[i], yf[i],
46              color='#b71c1c', linestyle=':', linewidth=1.6, zorder=2)
47
48 ax.set_xlabel('$x$')
49 ax.set_ylabel('$y$')
50 ax.set_title('OLS Fit on Synthetic Data with Injected Outliers')
51 ax.annotate(rf'MSE$_{\mathrm{{{train}}}}$ = {mse_train:.3f}' '\n'
52            rf'MSE$_{\mathrm{{{test}}}}$ = {mse_test:.3f}',
53            xy=(0.98, 0.03), xycoords='axes fraction',

```

```

54     ha='right', va='bottom', fontsize=10, linespacing=1.6,
55     bbox=dict(boxstyle='round,pad=0.5', facecolor='white',
56               edgecolor='#b0b0b0', alpha=0.9))
57 ax.legend(framealpha=0.9, loc='upper left')
58
59 plt.tight_layout()
60 save_fig('q1_2_ols_fit_outliers')
61 plt.show()

```

Table 1: Fitted parameters and mean squared errors of the OLS model.

Quantity	Value
Intercept w_0	3.3842
Slope w_1	1.8197
MSE_{train}	11.1480
MSE_{test}	9.8696

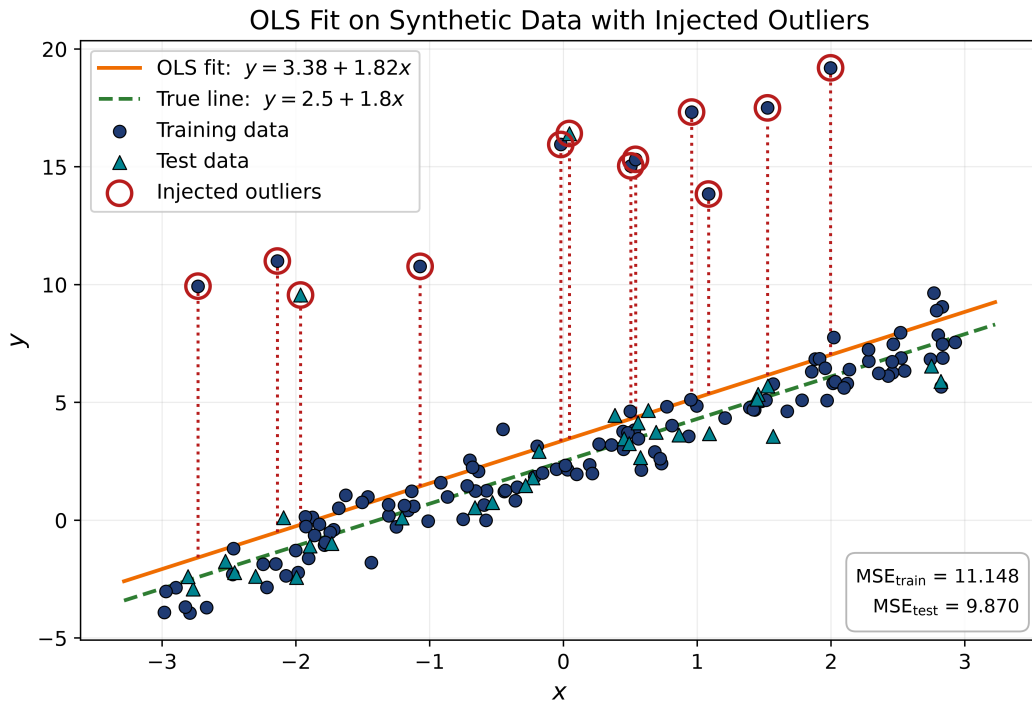


Figure 1: OLS fit on the synthetic data. The dotted vertical segments mark the residual of each injected outlier against the fitted line.

The fitted intercept of 3.3842 sits well above the true value of 2.5, while the slope 1.8197 remains close to the true 1.8. This is the expected signature of purely vertical contamination: the twelve upward shifts raise the fitted line bodily rather than tilting it.

1.3 Weighted Least Squares

Define $a_i \in (0, 1]$ as the sample-specific weight, where $a_i = 0.05$ for the known outliers and $a_i = 1.0$ for all inliers. Derive the closed-form matrix expression for the weighted least-squares estimator $\hat{\mathbf{w}}_{\text{WLS}} = (\mathbf{X}^T \mathbf{A} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{A} \mathbf{y}$. Implement this WLS estimator using NumPy and compare its fitted line against standard OLS.

Weighted Least Squares

We have

$$J(\mathbf{w}) = \frac{1}{2N} (\mathbf{y} - \mathbf{X}\mathbf{w})^T \mathbf{A} (\mathbf{y} - \mathbf{X}\mathbf{w}) \quad (1)$$

where $\mathbf{A}$ is a diagonal matrix containing the weights a_i . The known outliers have weight 0.05, while the inliers have weight 1.0.

1. Expanding the objective

Expanding the quadratic expression,

$$J(\mathbf{w}) = \frac{1}{2N} [\mathbf{y}^T \mathbf{A} \mathbf{y} - \mathbf{y}^T \mathbf{A} \mathbf{X} \mathbf{w} - \mathbf{w}^T \mathbf{X}^T \mathbf{A} \mathbf{y} + \mathbf{w}^T \mathbf{X}^T \mathbf{A} \mathbf{X} \mathbf{w}] .$$

Since $\mathbf{A}$ is diagonal, it is symmetric. The two middle terms are scalars and are equal, so this becomes

$$J(\mathbf{w}) = \frac{1}{2N} [\mathbf{y}^T \mathbf{A} \mathbf{y} - 2\mathbf{w}^T \mathbf{X}^T \mathbf{A} \mathbf{y} + \mathbf{w}^T \mathbf{X}^T \mathbf{A} \mathbf{X} \mathbf{w}] .$$

2. Gradient

Taking the derivative with respect to $\mathbf{w}$,

$$\nabla_{\mathbf{w}} J = \frac{1}{2N} [-2\mathbf{X}^T \mathbf{A} \mathbf{y} + 2\mathbf{X}^T \mathbf{A} \mathbf{X} \mathbf{w}] .$$

Therefore,

$$\nabla_{\mathbf{w}} J = \frac{1}{N} (\mathbf{X}^T \mathbf{A} \mathbf{X} \mathbf{w} - \mathbf{X}^T \mathbf{A} \mathbf{y}) . \quad (2)$$

3. Setting the gradient to zero

For the minimum, we set the gradient equal to zero:

$$\mathbf{X}^T \mathbf{A} \mathbf{X} \hat{\mathbf{w}} = \mathbf{X}^T \mathbf{A} \mathbf{y} . \quad (3)$$

These are the weighted normal equations.

4. Solving for $\hat{\mathbf{w}}$

All the weights α_i are positive, so $\mathbf{A}$ is positive definite. Assuming that $\mathbf{X}$ has full column rank, $\mathbf{X}^T \mathbf{A} \mathbf{X}$ is invertible.

Thus,

$$\hat{\mathbf{w}}_{\text{WLS}} = (\mathbf{X}^T \mathbf{A} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{A} \mathbf{y}. \quad (4)$$

5. Checking the minimum

The Hessian is

$$\nabla_{\mathbf{w}}^2 J = \frac{1}{N} \mathbf{X}^T \mathbf{A} \mathbf{X}.$$

Since $\mathbf{X}^T \mathbf{A} \mathbf{X}$ is positive definite, the Hessian is also positive definite. Therefore, the stationary point is the unique global minimum of $J(\mathbf{w})$.

Hence, the weighted least-squares solution is

$$\hat{\mathbf{w}}_{\text{WLS}} = (\mathbf{X}^T \mathbf{A} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{A} \mathbf{y}.$$

Listing 3: NumPy implementation of the OLS and WLS closed-form estimators.

```
1 # -----
2 # NumPy implementation of the OLS and WLS closed-form estimators
3 # -----
4
5 # Boolean outlier mask over the full dataset, then mapped onto each split
6 is_out = np.zeros(len(y), dtype=bool)
7 is_out[outlier_idx] = True
8
9 out_train = is_out[idx_train] # which training rows are outliers
10 out_test = is_out[idx_test] # which test rows are outliers
11
12 print(f"Outliers in training set: {out_train.sum()} / {len(out_train)}")
13 print(f"Outliers in test set : {out_test.sum()} / {len(out_test)}")
14
15
16 def design_matrix(x):
17     """Return X with the lead column of ones for the intercept bias."""
18     x = np.asarray(x).reshape(len(x), -1)
19     return np.column_stack([np.ones(len(x)), x])
20
21
22 Xd_train = design_matrix(X_train)
23 Xd_test = design_matrix(X_test)
24
25 # Sample-specific weights: a_i = 0.05 for known outliers, a_i = 1.0 for inliers
26 a = np.where(out_train, 0.05, 1.0)
27 A = np.diag(a) # diagonal weighting matrix
28
```



```

29 # OLS : w = (X^T X)^-1 X^T y      (the special case A = I)
30 # WLS : w = (X^T A X)^-1 X^T A y
31 # np.linalg.solve is the numerically stable way to evaluate these closed forms
32 w_ols = np.linalg.solve(Xd_train.T @ Xd_train, Xd_train.T @ y_train)
33 w_wls = np.linalg.solve(Xd_train.T @ A @ Xd_train, Xd_train.T @ A @ y_train)
34
35 # Check the closed-form OLS reproduces the sklearn fit from Q2
36 assert np.allclose(w_ols, [w0, w1]), "Closed-form OLS disagrees with sklearn"
37
38
39 def mse(Xd, y_true, w, mask=None):
40     """Mean squared error, optionally over a subset of rows."""
41     resid = y_true - Xd @ w
42     if mask is not None:
43         resid = resid[mask]
44     return float(np.mean(resid ** 2))
45
46
47 # Overall test MSE is dominated by the outliers WLS deliberately ignores, so the
48 # inlier-only MSE is reported too - that is what shows the benefit of weighting
49 print(f"\n{'Model':<6}{'w0':>9}{'w1':>9}{'MSE_train':>12}{'MSE_test':>11}"
50       f"{'MSE_test (inliers)':>21}")
51 for name, w in (("OLS", w_ols), ("WLS", w_wls)):
52     print(f"{name:<6}{w[0]:>9.4f}{w[1]:>9.4f}"
53           f"{mse(Xd_train, y_train, w):>12.4f}"
54           f"{mse(Xd_test, y_test, w):>11.4f}"
55           f"{mse(Xd_test, y_test, w, ~out_test):>21.4f}")
56 print(f"{'True':<6}{2.5:>9.4f}{1.8:>9.4f}")
57
58 print("\nEuclidean distance from the true parameters [2.5, 1.8]:")
59 print(f" ||w_ols - w_true|| = {np.linalg.norm(w_ols - [2.5, 1.8]):.4f}")
60 print(f" ||w_wls - w_true|| = {np.linalg.norm(w_wls - [2.5, 1.8]):.4f}")

```

Table 2: OLS against WLS with $a_i = 0.05$ on the known outliers. The inlier-only test MSE excludes the two contaminated test rows.

Model	w_0	w_1	MSE _{train}	MSE _{test}	MSE _{test} (inliers)
OLS	3.3842	1.8197	11.1480	9.8696	1.7663
WLS	2.4926	1.8394	11.9395	10.1487	0.6348
True	2.5000	1.8000	—	—	—

Outliers in training set: 10 / 128

Outliers in test set : 2 / 32

Euclidean distance from the true parameters [2.5, 1.8]:

$||w_{ols} - w_{true}|| = 0.8844$

$||w_{wls} - w_{true}|| = 0.0401$

Listing 4: Plotting the WLS and OLS fitted lines against the true line.

```

1 # -----
2 # WLS vs OLS fitted lines
3 # -----

```

```

4  Xf   = np.asarray(X).ravel()
5  X_tr = np.asarray(X_train).ravel()
6  X_te = np.asarray(X_test).ravel()
7
8  x_line = np.linspace(Xf.min() - 0.3, Xf.max() + 0.3, 200)
9
10 fig, ax = plt.subplots(figsize=(8, 5.5))
11
12 ax.plot(x_line, w_ols[0] + w_ols[1] * x_line, color='#ef6c00', linewidth=2,
13         label=r'OLS:  $y = \{w_{ols}[0] : .2f\} + \{w_{ols}[1] : .2f\}x$')
14 ax.plot(x_line, w_wls[0] + w_wls[1] * x_line, color='#1f3b73', linewidth=2,
15         label=r'WLS ($a_i = 0.05$):  $y = \{w_{wls}[0] : .2f\} + \{w_{wls}[1] : .2f\}x$')
16 ax.plot(x_line, 2.5 + 1.8 * x_line, color='#2e7d32', linewidth=2, linestyle='--',
17         label=r'True line:  $y = 2.5 + 1.8x$')
18
19 ax.scatter(X_tr[~out_train], y_train[~out_train], s=42, color='#546e7a',
20           zorder=3, edgecolor='black', linewidth=0.5, alpha=0.8,
21           label='Training inliers')
22 ax.scatter(X_te[~out_test], y_test[~out_test], s=52, marker='^',
23           color='#00838f', zorder=3, edgecolor='black', linewidth=0.5,
24           alpha=0.8, label='Test inliers')
25 ax.scatter(Xf[is_out], y[is_out], s=170, marker='X', color='#b71c1c',
26           zorder=4, edgecolor='black', linewidth=0.8,
27           label=r'Down-weighted outliers')
28
29 ax.set_xlabel('$x$')
30 ax.set_ylabel('$y$')
31 ax.set_title('Weighted vs. Ordinary Least Squares under Known Outliers')
32 ax.legend(framealpha=0.9, loc='upper left', fontsize=9.5)
33
34 plt.tight_layout()
35 save_fig('q1_3_wls_vs_ols')
36 plt.show()

```

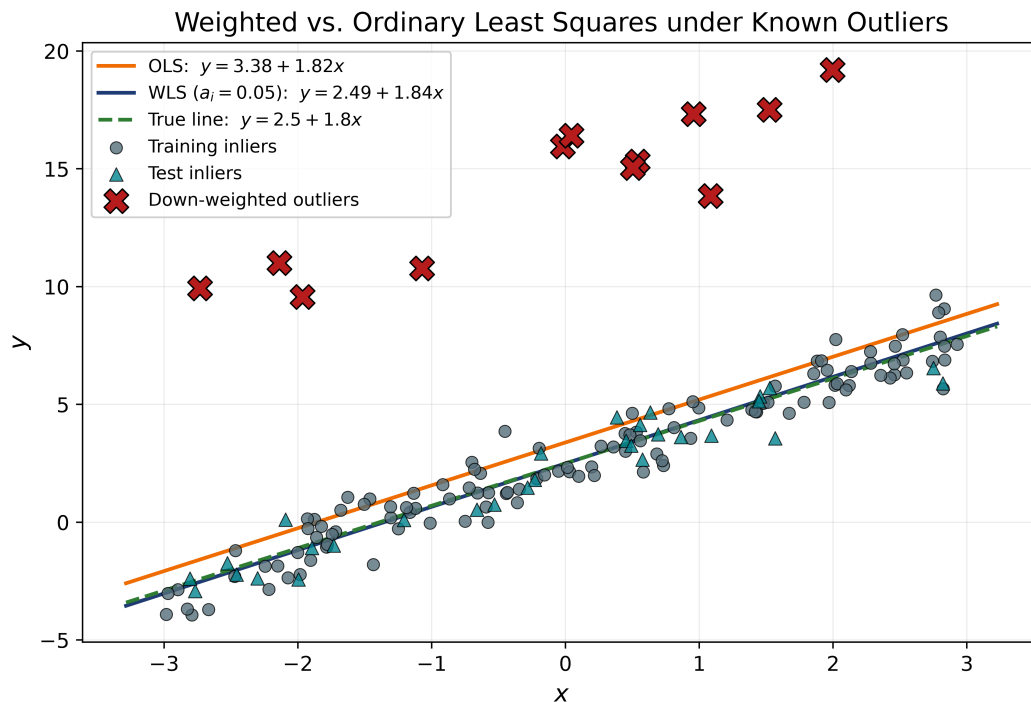


Figure 2: Weighted against ordinary least squares under known outliers. Down-weighting the twelve contaminated points recovers a line almost coincident with the true one.

Down-weighting the known outliers by a factor of twenty recovers an intercept of 2.4926 against the true 2.5, and reduces the distance from the true parameter vector from 0.8844 to 0.0401. The overall test MSE is marginally worse for WLS, because that metric is dominated by the two contaminated test rows the estimator deliberately discounts; on the inliers alone the test MSE falls from 1.7663 to 0.6348.

1.4 Regularization in High Dimensions

Create an expanded dataset containing nine pure noise variables. Fit OLS, Ridge Regression (L_2), and Lasso Regression (L_1). Tune the regularization strength hyperparameter $\alpha \in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10\}$ using 5-fold cross-validation. Report optimal α^ values and cross-validated test MSEs.*

Listing 5: OLS, Ridge and Lasso on one informative feature plus nine pure-noise features, with α tuned by 5-fold cross-validation.

```
1 # OLS / Ridge / Lasso on 1 informative + 9 pure-noise features
2
3 from sklearn.model_selection import GridSearchCV
4
5 # Nine pure noise variables appended to the single real feature. This draws from
6 # the same rng stream as Listing 1, so the cells must be run in order.
7 X_noise = rng.normal(size=(X.shape[0], 9))
8 X10 = np.column_stack([X, X_noise])      # 160 x 10: column 0 real, columns 1-9 noise
9
10 # Same 80:20 split and seed as Q2, so the two questions stay comparable
```

```

11 X10_train, X10_test, y10_train, y10_test = train_test_split(
12     X10, y, test_size=0.20, random_state=42)
13 assert np.array_equal(y10_train, y_train) # confirms it is the same partition
14
15 alphas = [1e-3, 1e-2, 1e-1, 1.0, 10.0] # regularization strengths to search
16
17 # Each estimator sits behind a StandardScaler in a Pipeline, so the scaler is
18 # refitted inside every CV fold - fitting it once on the whole training set
19 # would leak validation-fold statistics into training. OLS has no alpha to tune.
20 models = {
21     'OLS': (LinearRegression(), {}),
22     'Ridge': (Ridge(), {'reg__alpha': alphas}),
23     'Lasso': (Lasso(max_iter=50000), {'reg__alpha': alphas}),
24 }
25
26 rows, coefs = [], {}
27 for name, (est, grid) in models.items():
28     pipe = Pipeline([('scale', StandardScaler()), ('reg', est)])
29     search = GridSearchCV(pipe, grid, cv=5, scoring='neg_mean_squared_error')
30     search.fit(X10_train, y10_train)
31
32     best_alpha = search.best_params_.get('reg__alpha') # None for OLS
33     cv_mse = -search.best_score_ # mean 5-fold CV MSE
34     test_mse = mean_squared_error(y10_test, search.predict(X10_test))
35
36     # Clip numerically tiny values to exactly zero so Lasso's dropped features
37     # are unambiguous
38     w = search.best_estimator_.named_steps['reg'].coef_
39     w = np.where(np.abs(w) < 1e-8, 0.0, w)
40     coefs[name] = w
41     n_zero = int(np.sum(w[1:] == 0.0)) # noise coefficients driven to zero
42     rows.append((name, best_alpha, cv_mse, test_mse, n_zero))
43
44 print(f"{'Model':<8}{ 'alpha*':>9}{ 'CV MSE':>11}{ 'Test MSE':>11}{ 'noise coefs = 0':>18}")
45 for name, alpha, cv_mse, test_mse, n_zero in rows:
46     alpha_str = '-' if alpha is None else f'{alpha:g}'
47     print(f"{'name':<8}{alpha_str:>9}{cv_mse:>11.4f}{test_mse:>11.4f}"
48           f"{'{n_zero} / 9':>18}")
49
50 # Coefficients on the standardized features, so magnitudes are comparable
51 print("\nFitted coefficients (feature 0 is the real one, 1-9 are pure noise):")
52 print(f"{'':<8}" + ''.join(f'{i:>8}' for i in range(10)))
53 for name in models:
54     print(f"{'name':<8}" + ''.join(f'{c:>8.3f}' for c in coefs[name]))

```

Table 3: Optimal regularization strengths and cross-validated errors on the expanded ten-feature dataset.

Model	α^*	CV MSE	Test MSE	Noise coefs = 0
OLS	—	14.0434	10.3007	0 / 9
Ridge	10	13.7966	10.0921	0 / 9
Lasso	1	12.3842	10.1284	9 / 9

Fitted coefficients (feature 0 is the real one, 1-9 are pure noise):

	0	1	2	3	4	5	6	7	8	9
OLS	3.165	-0.026	0.070	-0.078	-0.157	-0.486	0.150	-0.329	0.062	-0.332
Ridge	2.922	-0.008	0.057	-0.055	-0.128	-0.419	0.138	-0.295	0.078	-0.331
Lasso	2.111	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000

Listing 6: Bar chart of the fitted coefficients across the ten features.

```

1 # Coefficient magnitudes: real feature vs. the nine noise features
2
3 fig, ax = plt.subplots(figsize=(9.5, 5))
4
5 idx = np.arange(10)
6 width = 0.27
7
8 for i, (name, color) in enumerate(zip(('OLS', 'Ridge', 'Lasso'),
9                                     ('#ef6c00', '#1f3b73', '#2e7d32'))):
10     ax.bar(idx + (i - 1) * width, coefs[name], width, label=name,
11           color=color, edgecolor='black', linewidth=0.5, zorder=3)
12
13 ax.axhline(0, color='black', linewidth=0.9, zorder=2)
14 ax.axvline(0.5, color='b71c1c', linestyle='--', linewidth=1.3, zorder=2)
15
16 ax.set_xticks(idx)
17 ax.set_xticklabels(['x$\n(real)'] + [f'$n_{i}$' for i in range(1, 10)])
18 ax.set_xlabel('Feature')
19 ax.set_ylabel('Fitted coefficient (standardized features)')
20 ax.set_title('Coefficients on 1 Informative + 9 Pure-Noise Features')
21 ax.legend(framealpha=0.9, loc='upper right')
22
23 plt.tight_layout()
24 save_fig('q1_4_coefficients')
25 plt.show()

```

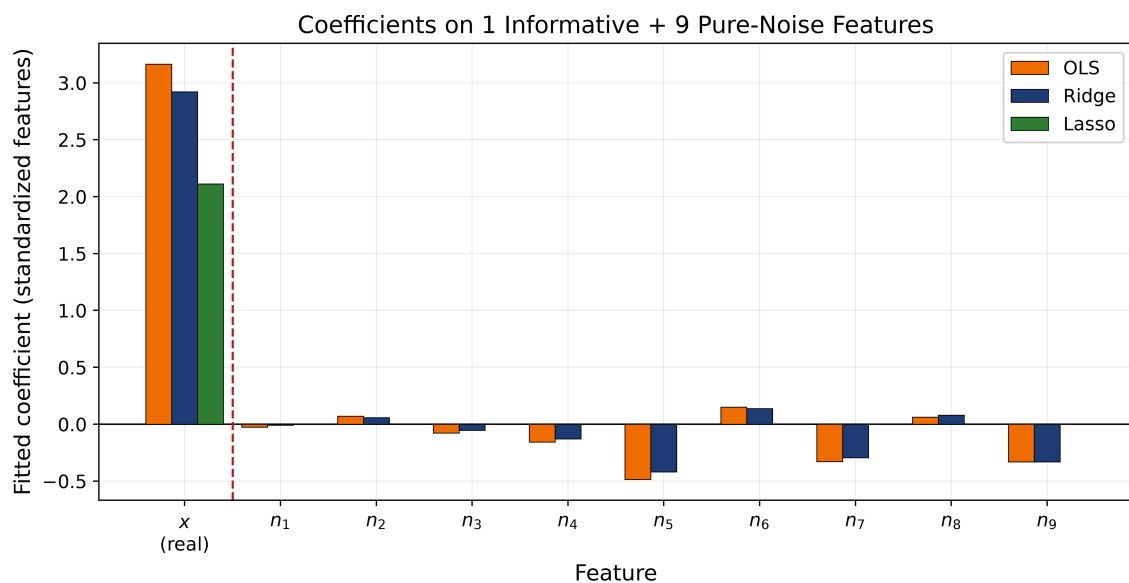


Figure 3: Fitted coefficients on one informative and nine pure-noise features. Lasso eliminates every noise coefficient exactly; Ridge shrinks them but retains all nine.

1.5 Ridge against Lasso

Compare Ridge and Lasso in terms of their mathematical penalty terms ($\|\mathbf{w}\|_2^2$ vs. $\|\mathbf{w}\|_1$), resulting coefficient magnitudes, and capacity for sparse feature selection. Explain which model is preferable when $D \gg N$ and only a small subset of features are genuinely informative.

Ridge and Lasso both add a penalty to the loss function, but they affect the model coefficients in different ways.

Ridge uses the L_2 penalty:

$$\alpha \|\mathbf{w}\|_2^2 = \alpha \sum_j w_j^2 \quad (5)$$

This penalty is smooth and differentiable. Lasso uses the L_1 penalty:

$$\alpha \|\mathbf{w}\|_1 = \alpha \sum_j |w_j| \quad (6)$$

The L_1 penalty is not differentiable at $w_j = 0$.

For an orthonormal design where

$$\mathbf{X}^T \mathbf{X} = \mathbf{I},$$

and ρ_j is the OLS coefficient, Ridge gives

$$\hat{w}_j = \frac{\rho_j}{1 + \lambda}. \quad (7)$$

Therefore, Ridge shrinks all the coefficients towards zero by the same factor. But the coefficients normally do not become exactly zero.

For Lasso, the solution is

$$\hat{w}_j = \text{sign}(\rho_j) \max \left(|\rho_j| - \frac{\lambda}{2}, 0 \right). \quad (8)$$

If the value of $|\rho_j|$ is smaller than or equal to $\lambda/2$, the coefficient becomes exactly zero.

The difference can also be understood geometrically. The L_2 constraint gives a round constraint region, so the loss contours usually touch it at a point where most coefficients are nonzero. The L_1 constraint has corners along the axes, making it more likely for the optimum to occur where some coefficients are exactly zero.

Therefore, Lasso can automatically perform feature selection, while Ridge mainly shrinks the coefficients.

When $D \gg N$ and only a few features are actually informative, I would prefer Lasso. Since most of the features are irrelevant, Lasso can set their coefficients to zero and produce a sparse model. This also makes the model easier to interpret.

In our Q4 experiment, Lasso with the selected α^* kept only the true feature x and set the 9 noise-feature coefficients to zero. Ridge, on the other hand, kept all 10 coefficients nonzero. This matches the expected difference between the two methods.

So, for a high-dimensional dataset with only a few useful features, Lasso is generally more suitable because it can both reduce the effect of irrelevant features and select the important ones.

1.6 Necessity of Standardized Feature Scaling

Explain analytically why standardized feature scaling ($z = \frac{x-\mu}{\sigma}$) is mandatory prior to penalizing weights in Ridge or Lasso regression.

Feature scaling is important before applying Ridge or Lasso because the regularization penalty is applied directly to the model weights. If the features have very different scales, the weights can also have very different magnitudes even when the features are equally important.

Suppose a feature x_j is changed to a different unit:

$$x'_j = cx_j$$

To keep the predictions the same, its coefficient has to change to

$$w'_j = \frac{w_j}{c}. \tag{9}$$

For Ridge, the penalty for this coefficient changes from

$$\alpha w_j^2$$

to

$$\alpha (w'_j)^2 = \alpha \frac{w_j^2}{c^2}.$$

For Lasso, it changes from

$$\alpha |w_j|$$

to

$$\alpha|w'_j| = \alpha \frac{|w_j|}{c}.$$

Therefore, simply changing the units of a feature changes how strongly that feature is penalized. A feature with a small numerical scale will usually need a larger coefficient, so it can receive a stronger effective penalty. A feature with a large numerical scale can have a smaller coefficient and will be penalized less.

This is especially important for Lasso because the penalty can make a coefficient exactly zero. Therefore, the scale of a feature can affect whether that feature is selected or removed from the model.

To avoid this problem, we standardize each feature using

$$z_j = \frac{x_j - \mu_j}{\sigma_j}. \quad (10)$$

After standardization, the features have approximately zero mean and unit variance. This puts them on the same scale, so the same regularization parameter α can be applied fairly to all coefficients.

Therefore, standardization is needed before Ridge or Lasso when the features have different scales. It makes the effect of the regularization depend more on the actual importance of the features rather than on their measurement units.

This is also why the Q4 pipelines use `StandardScaler` before applying Ridge and Lasso.

2. Logistic Regression: Preprocessing, Interpretation, and Evaluation

The classification dataset contains 600 samples with four numerical features, two nominal categorical columns, deliberate class imbalance (68 % negative, 32 % positive), 3 % label noise, and 25 missing entries in `x3`.

Listing 7: Generation of the classification dataset with mixed attributes and missing values.

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.datasets import make_classification
4
5 # Four numeric features: 3 informative, 1 redundant (a linear combination of them)
6 X_num, y = make_classification(
7     n_samples=600,
8     n_features=4,
9     n_informative=3,
10    n_redundant=1,
```



```

11     class_sep=0.9,          # how far apart the two class clusters sit
12     weights=[0.68, 0.32],  # class imbalance: 68% negative, 32% positive
13     flip_y=0.03,           # 3% of Labels randomly flipped (Label noise)
14     random_state=21
15 )
16
17 df = pd.DataFrame(X_num, columns=["x1", "x2", "x3", "x4"])
18 rng = np.random.default_rng(21)
19 df["region"] = rng.choice(["north", "south", "east"], size=len(df)) # nominal, 3 levels
20 df["device"] = rng.choice(["mobile", "desktop"], size=len(df))      # nominal, 2 levels
21 df.loc[rng.choice(len(df), 25, replace=False), "x3"] = np.nan      # 25 missing values

```

2.1 String-Valued Columns and Label Encoding

Explain why passing string-valued columns directly to LogisticRegression throws a run-time error. Furthermore, explain why applying basic LabelEncoder mapping categories to integers $\{0, 1, 2\}$ introduces a spurious ordinal assumption that degrades model generalizability.

Logistic Regression cannot directly use string values such as "north", "south", "east", "mobile", or "desktop". The model works with numerical values. It first calculates a linear score

$$z = \mathbf{w}^T \mathbf{x} + b \quad (11)$$

and then applies the sigmoid function

$$p = \frac{1}{1 + e^{-z}} \quad (12)$$

Because of this, the input features need to be numerical. If string columns are given directly to LogisticRegression, scikit-learn tries to convert the values into numbers. Values such as "north" cannot be converted, so an error like the following can occur:

```
ValueError: could not convert string to float: 'north'
```

Using LabelEncoder is also not a good solution for nominal categorical features. For example, suppose the region values are encoded as

```

east  -> 0
north -> 1
south -> 2

```

Logistic Regression would then treat these values as numerical quantities. This creates an artificial ordering:

$$\text{south} > \text{north} > \text{east}.$$

It also assumes that the distance between east and north is the same as the distance between north and south.

There is no such relationship between these regions. They are nominal categories, so there is no natural order or numerical distance between them. This is known as a spurious ordinal assumption.

Because of this, the model could learn patterns based on the arbitrary numbers assigned by LabelEncoder. Also, changing the encoding, for example by assigning east -> 2 instead of east -> 0, could change the model even though the original data has not changed.

2.2 Preprocessing Pipeline and Stratified Split

Construct a scikit-learn Pipeline combined with ColumnTransformer that imputes missing numerical entries with training set column medians; standardizes numerical features ($\mu = 0$, $\sigma = 1$); imputes missing categorical features using the most frequent mode; one-hot encodes categorical attributes; and fits a Logistic Regression classifier. Split the dataset into training (80 %) and test (20 %) subsets using a stratified split with random_state=42.

Listing 8: Preprocessing pipeline combining ColumnTransformer with imputation, scaling, one-hot encoding and the classifier.

```
1 from sklearn.compose import ColumnTransformer
2 from sklearn.impute import SimpleImputer
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.model_selection import train_test_split
5 from sklearn.pipeline import Pipeline
6 from sklearn.preprocessing import OneHotEncoder, StandardScaler
7
8 num_cols = ["x1", "x2", "x3", "x4"] # numerical features
9 cat_cols = ["region", "device"] # categorical features
10
11 X = df[num_cols + cat_cols]
12
13 # Stratified 80:20 split, so both sets keep the same class balance
14 X_train, X_test, y_train, y_test = train_test_split(
15     X, y, test_size=0.20, stratify=y, random_state=42)
16
17 # Numerical: fill missing with the median, then standardize
18 num_pipe = Pipeline([
19     ("impute", SimpleImputer(strategy="median")),
20     ("scale", StandardScaler()),
21 ])
22
23 # Categorical: fill missing with the mode, then one-hot encode
24 cat_pipe = Pipeline([
25     ("impute", SimpleImputer(strategy="most_frequent")),
26     ("onehot", OneHotEncoder(handle_unknown="ignore")),
27 ])
28
29 # Apply each sub-pipeline to its own set of columns
30 preprocessor = ColumnTransformer([
```

```

31     ("num", num_pipe, num_cols),
32     ("cat", cat_pipe, cat_cols),
33 ])
34
35 # Full pipeline: preprocessing + classifier
36 clf = Pipeline([
37     ("pre", preprocessor),
38     ("model", LogisticRegression(max_iter=1000)),
39 ])
40
41 clf.fit(X_train, y_train)
42
43 print(f"Train size: {X_train.shape[0]}, test size: {X_test.shape[0]}")
44 print(f"Positive class share - train: {y_train.mean():.3f}, test: {y_test.mean():.3f}")
45 print(f"Missing x3 values    - train: {X_train['x3'].isna().sum()}, "
46       f"test: {X_test['x3'].isna().sum()}")
47
48 # Column names after one-hot encoding
49 print("\nFeatures after preprocessing:")
50 print(list(clf.named_steps["pre"].get_feature_names_out()))
51
52 print(f"\nTrain accuracy: {clf.score(X_train, y_train):.4f}")
53 print(f"Test accuracy : {clf.score(X_test, y_test):.4f}")

```

```

Train size: 480, test size: 120
Positive class share - train: 0.329, test: 0.325
Missing x3 values    - train: 22, test: 3

Features after preprocessing:
['num__x1', 'num__x2', 'num__x3', 'num__x4', 'cat__region_east',
 'cat__region_north', 'cat__region_south', 'cat__device_desktop',
 'cat__device_mobile']

Train accuracy: 0.8854
Test accuracy : 0.8500

```

Nesting the imputer inside the pipeline is what makes the medians training-set medians as the question requires: the median is computed during `fit` on the 480 training rows and merely applied to the test set, so no test information leaks into training. The stratified split holds the positive-class share at 0.329 and 0.325 against the population rate of 0.328.

2.3 Grid Search over Regularization Strength and Penalty

Perform a grid search over regularization strengths $C \in \{0.01, 0.1, 1, 10, 100\}$ and penalty types $\text{penalty} \in \{\ell_1, \ell_2\}$ using `solver='liblinear'` and 5-fold Stratified Cross-Validation. Optimize for F_1 -score. Report best parameters and cross-validation score.

Listing 9: Grid search over C and penalty type with 5-fold stratified cross-validation, optimizing F_1 .

```

1 import warnings
2
3 from sklearn.model_selection import GridSearchCV, StratifiedKFold
4

```

```

5 # sklearn 1.8+ deprecates penalty= in favour of L1_ratio, but Q3 asks for
6 # penalty in {L1, L2}, so keep penalty= and hide the deprecation notices
7 warnings.filterwarnings("ignore", message="'penalty' was deprecated")
8 warnings.filterwarnings("ignore", message="Inconsistent values: penalty")
9
10 # Same pipeline as Q2, but with Liblinear so both L1 and L2 are supported
11 grid_clf = Pipeline([
12     ("pre", preprocessor),
13     ("model", LogisticRegression(solver="liblinear", max_iter=1000)),
14 ])
15
16 param_grid = {
17     "model__C": [0.01, 0.1, 1, 10, 100],
18     "model__penalty": ["l1", "l2"],
19 }
20
21 # 5-fold stratified CV keeps the class balance in every fold
22 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
23
24 search = GridSearchCV(grid_clf, param_grid, cv=cv, scoring="f1")
25 search.fit(X_train, y_train)
26
27 best_model = search.best_estimator_ # refit on the full training set
28
29 print(f"Best parameters : C = {search.best_params_['model__C']}, "
30       f"penalty = {search.best_params_['model__penalty']}")
31 print(f"Best CV F1-score: {search.best_score_:.4f}")
32
33 # Mean CV F1 for every combination in the grid
34 print(f"\nC:{>8}{l1:>10}{l2:>10}")
35 results = search.cv_results_
36 for C in param_grid["model__C"]:
37     scores = []
38     for penalty in ("l1", "l2"):
39         i = next(k for k, p in enumerate(results["params"])
40                 if p["model__C"] == C and p["model__penalty"] == penalty)
41         scores.append(results["mean_test_score"][i])
42     print(f"C:{>8}{scores[0]:>10.4f}{scores[1]:>10.4f}")

```

Table 4: Mean 5-fold cross-validated F_1 -score for every combination in the grid.

C	ℓ_1	ℓ_2
0.01	0.7418	0.7843
0.1	0.8007	0.8033
1	0.8148	0.8073
10	0.7997	0.7997
100	0.7997	0.7997

The best parameters are $C = 1$ with an ℓ_1 penalty, giving a best cross-validated F_1 -score of **0.8148**. The scores at $C = 10$ and $C = 100$ are identical for both penalties, indicating that regularization has effectively switched off at that strength and both penalties converge to the same unconstrained solution.

2.4 Test-Set Evaluation and Class Imbalance

On the test set, evaluate and report accuracy, precision, recall, F_1 -score, and confusion matrix. Explain why overall accuracy can be deeply misleading when dealing with class-imbalanced datasets (e.g., 68 % negative, 32 % positive).

Listing 10: Test-set evaluation of the tuned classifier.

```
1 from sklearn.metrics import (ConfusionMatrixDisplay, accuracy_score,
2                               confusion_matrix, f1_score, precision_score,
3                               recall_score)
4
5 # Predict with the best model found by the Q3 grid search
6 y_pred = best_model.predict(X_test)
7
8 print(f"Accuracy : {accuracy_score(y_test, y_pred):.4f}")
9 print(f"Precision: {precision_score(y_test, y_pred):.4f}")
10 print(f"Recall   : {recall_score(y_test, y_pred):.4f}")
11 print(f"F1-score : {f1_score(y_test, y_pred):.4f}")
12
13 cm = confusion_matrix(y_test, y_pred)
14 print(f"\nConfusion matrix:\n{cm}")
15
16 tn, fp, fn, tp = cm.ravel()
17 print(f"\nTN = {tn}, FP = {fp}, FN = {fn}, TP = {tp}")
18
19 # Accuracy of a baseline that always predicts the majority (negative) class
20 baseline = 1 - y_test.mean()
21 print(f"\nMajority-class baseline accuracy: {baseline:.4f}")
```

Listing 11: Rendering the confusion matrix.

```
1 fig, ax = plt.subplots(figsize=(5.5, 5))
2
3 ConfusionMatrixDisplay(cm, display_labels=["negative (0)", "positive (1)"]).plot(
4     ax=ax, cmap="Blues", colorbar=False)
5
6 ax.grid(False)      # the global grid setting would draw over the cells
7 ax.set_title("Confusion Matrix (Test Set)")
8
9 plt.tight_layout()
10 save_fig("q2_4_confusion_matrix")
11 plt.show()
```

Test-set results:

Table 5: Test-set performance of the tuned logistic regression model.

Metric	Value
Accuracy	0.8583
Precision	0.7895
Recall	0.7692
F1-score	0.7792

Confusion matrix (rows = true, columns = predicted):

Table 6: Confusion matrix on the 120 test samples.

	pred 0	pred 1
true 0	TN = 73	FP = 8
true 1	FN = 9	TP = 30

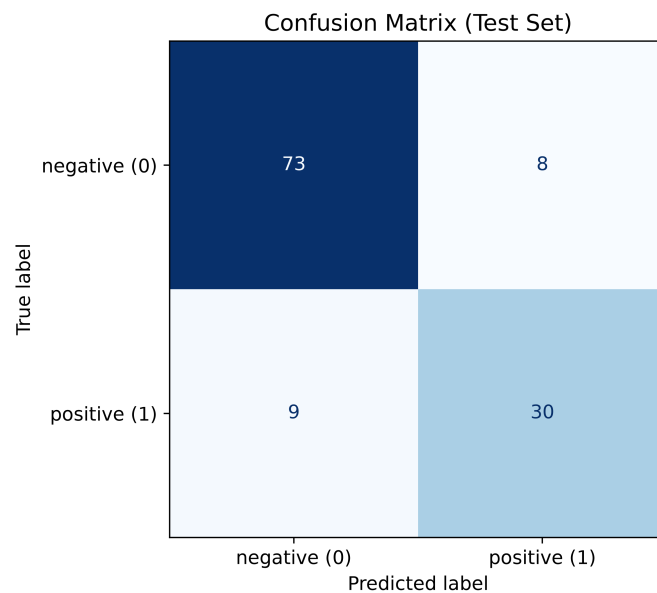


Figure 4: Confusion matrix of the tuned classifier on the test set.

Majority-class (always-0) baseline accuracy: **0.6750**.

Why overall accuracy is misleading under class imbalance

In this dataset, the classes are not balanced: about 68 % of the samples belong to the negative class and 32 % belong to the positive class. Because of this, accuracy alone does not give a good idea of how well the classifier is performing.

First, consider a very simple classifier that always predicts the majority class. There are 81 negative samples out of 120, so this classifier would get

$$\frac{81}{120} = 0.675 \quad (13)$$

Therefore, an accuracy of around 0.70 or even 0.80 does not necessarily mean that the model is learning useful patterns.

Another problem is that the majority class has a large effect on the accuracy. Accuracy is calculated as

$$\text{Accuracy} = \frac{TP + TN}{N} \quad (14)$$

Since the negative class makes up most of the test set, getting the negative predictions correct can make the accuracy look good. For example, a model that completely fails to detect the positive class and has a recall of 0 could still get 67.5 % accuracy by predicting every sample as negative.

Accuracy also treats false positives and false negatives in the same way. However, these two types of errors may not have the same importance. For example, in medical diagnosis, missing a positive case can be much more serious than incorrectly identifying a negative case as positive. Accuracy does not show this difference.

For our model, the accuracy is 0.8583, which is much better than the 0.675 majority-class baseline. The model correctly identifies 30 out of the 39 positive samples. However, imagine that it detected only about 21 positive samples instead, with around 18 false negatives. The accuracy would still be about

$$\frac{21 + (81 - 8)}{120} \approx 0.78,$$

while the positive-class recall would drop to

$$\frac{21}{21 + 18} \approx 0.54.$$

So the accuracy would still look reasonably good even though the model would be missing almost half of the positive cases.

Therefore, accuracy should not be used alone for this imbalanced dataset. Measures such as precision, recall, F1-score, the confusion matrix, and ROC/PR curves give a better picture of the model's performance. This is also why the hyperparameter search in Q2.3 used F1-score instead of accuracy.

2.5 Decision Threshold and the Precision-Recall Trade-off

Adjust the classification decision boundary threshold from $p_{\text{threshold}} = 0.50$ down to 0.35. Recompute precision and recall. Describe the Precision-Recall trade-off and outline a realistic domain application (e.g., medical diagnostics, fraud detection) where lowering the threshold is preferred.

Listing 12: Recomputing precision and recall at the lowered decision threshold.

```
1 # Predicted probability of the positive class
2 y_proba = best_model.predict_proba(X_test)[: , 1]
3
4 # Default predict() is the same as thresholding at 0.50
5 assert np.array_equal((y_proba >= 0.50).astype(int), best_model.predict(X_test))
6
7
8 def metrics_at(t):
9     """Precision, recall, F1 and confusion matrix at threshold t."""
10    pred = (y_proba >= t).astype(int)
11    return (precision_score(y_test, pred), recall_score(y_test, pred),
12            f1_score(y_test, pred), confusion_matrix(y_test, pred))
13
14
15 print(f"{'Threshold':>10}{ 'Precision':>11}{ 'Recall':>9}{ 'F1':>9}"
16       f"{'TP':>6}{ 'FP':>5}{ 'FN':>5}{ 'TN':>5}")
17 for t in (0.50, 0.35):
18     prec, rec, f1, cm_t = metrics_at(t)
19     tn, fp, fn, tp = cm_t.ravel()
20     print(f"{t:>10.2f}{prec:>11.4f}{rec:>9.4f}{f1:>9.4f}"
21           f"{'tp':>6}{fp:>5}{fn:>5}{tn:>5}")
22
23 # Change from 0.50 to 0.35
24 p50, r50, _, _ = metrics_at(0.50)
25 p35, r35, _, _ = metrics_at(0.35)
26 print(f"\nPrecision change: {p50:.4f} -> {p35:.4f} ({p35 - p50:+.4f})")
27 print(f"Recall change   : {r50:.4f} -> {r35:.4f} ({r35 - r50:+.4f})")
```

Table 7: Effect of lowering the decision threshold from 0.50 to 0.35.

Threshold	Precision	Recall	F1	TP	FP	FN	TN
0.50	0.7895	0.7692	0.7792	30	8	9	73
0.35	0.6875	0.8462	0.7586	33	15	6	66

```
Precision change: 0.7895 -> 0.6875 (-0.1020)
Recall change   : 0.7692 -> 0.8462 (+0.0769)
```

Listing 13: Plotting the precision-recall curve with both operating points marked.

```
1 from sklearn.metrics import precision_recall_curve
2
3 prec_curve, rec_curve, _ = precision_recall_curve(y_test, y_proba)
```



```

4
5 fig, ax = plt.subplots(figsize=(7.5, 5.5))
6
7 ax.plot(rec_curve, prec_curve, color='#1f3b73', linewidth=2,
8         label='Precision-Recall curve')
9
10 # Mark the two operating points
11 for t, color, marker in ((0.50, '#ef6c00', 'o'), (0.35, '#b71c1c', 'x')):
12     prec, rec, _, _ = metrics_at(t)
13     ax.scatter(rec, prec, s=150, color=color, marker=marker, zorder=4,
14               edgecolor='black', linewidth=0.8,
15               label=rf'$p_{\{th\}} = \{t:.2f\}$: P = {prec:.3f}, R = {rec:.3f}')
16
17 # A classifier with no skill sits at the positive-class rate
18 ax.axhline(y_test.mean(), color='#546e7a', linestyle=':', linewidth=1.5,
19            label=f'No-skill baseline ({y_test.mean():.3f})')
20
21 ax.set_xlabel('Recall')
22 ax.set_ylabel('Precision')
23 ax.set_title('Precision-Recall Trade-off on the Test Set')
24 ax.set_ylim(0, 1.05)
25 ax.legend(framealpha=0.9, loc='lower left', fontsize=9.5)
26
27 plt.tight_layout()
28 save_fig('q2_5_precision_recall')
29 plt.show()

```

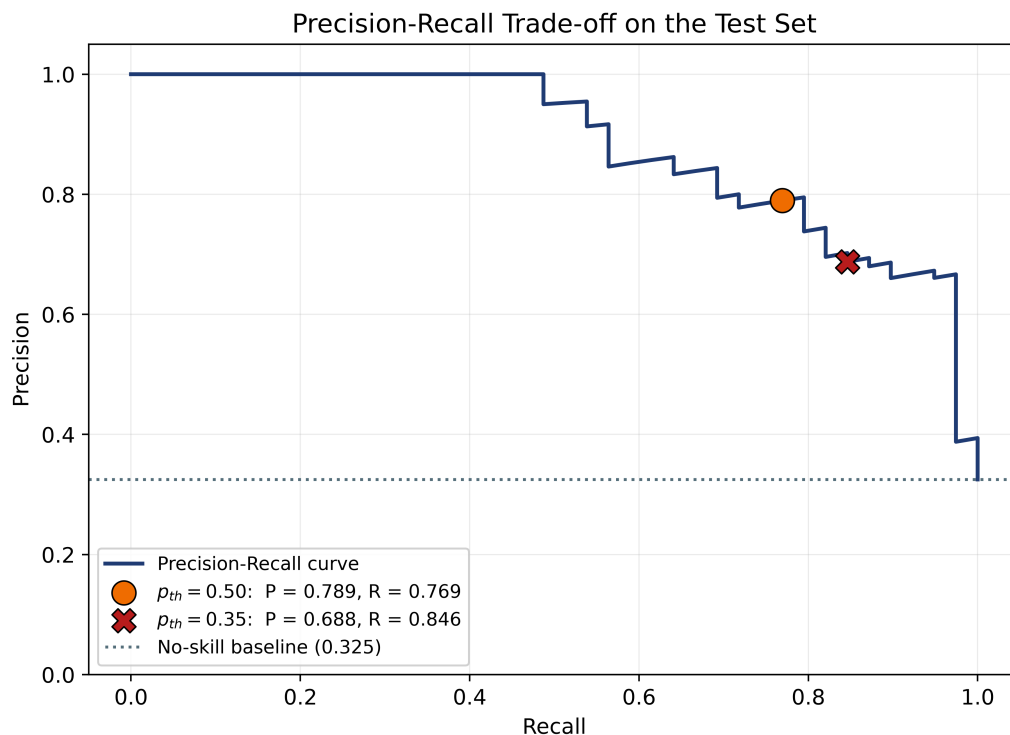


Figure 5: Precision-recall trade-off on the test set, with the $p_{th} = 0.50$ and $p_{th} = 0.35$ operating points marked.

When the threshold is reduced from 0.50 to 0.35, the model becomes less strict about predicting the positive class. This means that more samples are classified as positive.

Because of this, the model can identify more of the actual positive samples, so recall increases. However, some of the additional positive predictions will be incorrect, which causes precision to decrease.

So there is a trade-off between precision and recall. In general, increasing recall means accepting more false positives, which can reduce precision.

A good example is medical screening, such as detecting cancer or another serious disease. In this situation, a false negative can be much more serious than a false positive. Missing a patient who actually has the disease could delay treatment, while a false positive can usually be handled with another test.

Therefore, in this type of application, it can be useful to lower the decision threshold. This increases recall and helps reduce the number of patients who are missed, even though it also produces more false alarms.

2.6 Direct Probability Computation

A logistic regression model predicting whether a student passes with an A^+ ($y = 1$) yields $\ln\left(\frac{p}{1-p}\right) = -7.2 + 0.08x_1 + 1.4x_2$, where x_1 denotes study hours, x_2 denotes GPA, and $p = P(y = 1 \mid x)$. **(a)** Derive the sigmoid expression $p = \frac{1}{1+e^{-z}}$ from logit $z = \ln\left(\frac{p}{1-p}\right)$. Calculate the probability p for a student with $x_1 = 45$ study hours and $x_2 = 3.4$ GPA. **(b)** For a target probability $p = 0.70$ and GPA $x_2 = 3.4$, calculate the required study hours x_1 .

Deriving the sigmoid function:

$$\ln\left(\frac{p}{1-p}\right) = z$$

Taking e to both sides,

$$\frac{p}{1-p} = e^z$$

Multiplying both sides by $(1-p)$,

$$p = e^z(1-p)$$

Expanding,

$$p = e^z - e^z p$$

Moving the p terms to the same side,

$$p + e^z p = e^z$$

Factoring out p ,

$$p(1 + e^z) = e^z$$

Therefore,

$$p = \frac{e^z}{1 + e^z}$$

Dividing the numerator and denominator by e^z ,

$$p = \frac{1}{1 + e^{-z}} \quad (15)$$

(a) Probability for $x_1 = 45$ and $x_2 = 3.4$

The logistic model is

$$z = -7.2 + 0.08x_1 + 1.4x_2.$$

Substituting $x_1 = 45$ and $x_2 = 3.4$,

$$z = -7.2 + 0.08(45) + 1.4(3.4)$$

$$z = -7.2 + 3.6 + 4.76 = 1.16.$$

The logistic function is

$$p = \frac{1}{1 + e^{-z}}.$$

Therefore,

$$p = \frac{1}{1 + e^{-1.16}}$$

$$p \approx 0.761.$$

So, the predicted probability is approximately 0.76, or 76 %.

(b) Finding x_1 when $p = 0.70$

The relationship between the probability and the linear score is

$$z = \ln \left(\frac{p}{1-p} \right).$$

For $p = 0.70$,

$$z = \ln \left(\frac{0.70}{0.30} \right)$$

$$z \approx 0.847.$$

Using the model,

$$0.847 = -7.2 + 0.08x_1 + 1.4(3.4).$$

Since

$$1.4(3.4) = 4.76,$$

we get

$$0.847 = -7.2 + 0.08x_1 + 4.76.$$

Rearranging,

$$0.08x_1 = 0.847 + 7.2 - 4.76$$

$$0.08x_1 = 3.287.$$

Therefore,

$$x_1 = \frac{3.287}{0.08} \approx 41.1.$$

So, the student would need approximately 41.1 study hours for the model to predict a probability of 70 %.

Listing 14: Numerical check of the two hand calculations.

1 # Numerical check of the hand calculations

```

2  b0, b1, b2 = -7.2, 0.08, 1.4
3
4  # (a) p at x1 = 45 study hours, x2 = 3.4 GPA
5  z = b0 + b1 * 45 + b2 * 3.4
6  p = 1 / (1 + np.exp(-z))
7  print(f"(a) z = {z:.4f}, p = {p:.4f}")
8
9  # (b) study hours needed for p = 0.70 at x2 = 3.4
10 z_target = np.log(0.70 / (1 - 0.70)) # Logit of the target probability
11 x1 = (z_target - b0 - b2 * 3.4) / b1
12 print(f"(b) z = {z_target:.4f}, x1 = {x1:.4f} hours")

```

```

(a) z = 1.1600, p = 0.7613
(b) z = 0.8473, x1 = 41.0912 hours

```

3. Optimization of Logistic Regression

Let $\mathbf{X} \in \mathbb{R}^{N \times (D+1)}$ be the augmented feature matrix including a bias column of ones, $\mathbf{y} \in \{0, 1\}^N$ be target labels, $\mathbf{w} \in \mathbb{R}^{D+1}$ be model parameters, and

$$p_i = \sigma(\mathbf{w}^T \mathbf{x}_i) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}_i}}. \quad (16)$$

High-level estimators such as `sklearn.linear_model.LogisticRegression` are not used for the questions in this section.

Listing 15: Binary dataset for the gradient optimization algorithms.

```

1  import numpy as np
2  from sklearn.datasets import make_blobs
3  from sklearn.model_selection import train_test_split
4  from sklearn.preprocessing import StandardScaler
5
6  # Two Gaussian blobs, one per class
7  X, y = make_blobs(
8      n_samples=1200,
9      centers=[[-2.2, -0.5], [2.0, 1.2]],
10     cluster_std=[1.45, 1.65],
11     random_state=8
12 )
13
14 # Stratified 80:20 split
15 X_train, X_test, y_train, y_test = train_test_split(
16     X, y, test_size=0.2, stratify=y, random_state=42
17 )
18
19 print(f"Train: {X_train.shape}, test: {X_test.shape}")
20 print(f"Positive class share - train: {y_train.mean():.3f}, test: {y_test.mean():.3f}")

```

3.1 Gradient and Hessian of the Binary Cross-Entropy Loss

Derive the gradient vector $\nabla J(\mathbf{w})$ and Hessian matrix $\mathbf{H}$ for Binary Cross-Entropy loss

$$J(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)].$$

$$J(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)], \quad p_i = \sigma(z_i), \quad z_i = \mathbf{w}^T \mathbf{x}_i$$

with $\mathbf{x}_i \in \mathbb{R}^{D+1}$ including the lead 1 for the bias, and $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

Gradient

Per-sample loss $\ell_i = -y_i \ln p_i - (1 - y_i) \ln(1 - p_i)$. Differentiating w.r.t. p_i :

$$\frac{\partial \ell_i}{\partial p_i} = -\frac{y_i}{p_i} + \frac{1 - y_i}{1 - p_i}$$

Chaining through $p_i = \sigma(z_i)$ and $z_i = \mathbf{w}^T \mathbf{x}_i$, where $\frac{\partial p_i}{\partial \mathbf{w}} = p_i(1 - p_i)\mathbf{x}_i$:

$$\nabla J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \left[-\frac{y_i}{p_i} + \frac{1 - y_i}{1 - p_i} \right] p_i(1 - p_i) \mathbf{x}_i$$

The bracket collapses:

$$\left[-\frac{y_i}{p_i} + \frac{1 - y_i}{1 - p_i} \right] p_i(1 - p_i) = -y_i(1 - p_i) + (1 - y_i)p_i = p_i - y_i$$

$$\boxed{\nabla J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (p_i - y_i) \mathbf{x}_i = \frac{1}{N} \mathbf{X}^T (\mathbf{p} - \mathbf{y})} \quad (17)$$

where $\mathbf{p} = \sigma(\mathbf{X}\mathbf{w})$ elementwise.

Hessian

Only p_i depends on $\mathbf{w}$, and $\frac{\partial p_i}{\partial \mathbf{w}^T} = p_i(1 - p_i)\mathbf{x}_i^T$, so

$$\mathbf{H} = \frac{\partial}{\partial \mathbf{w}^T} \left[\frac{1}{N} \sum_{i=1}^N (p_i - y_i) \mathbf{x}_i \right] = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i \frac{\partial p_i}{\partial \mathbf{w}^T}$$

$$\boxed{\mathbf{H} = \frac{1}{N} \sum_{i=1}^N p_i(1 - p_i) \mathbf{x}_i \mathbf{x}_i^T = \frac{1}{N} \mathbf{X}^T \mathbf{S} \mathbf{X}} \quad (18)$$

with $\mathbf{S} = \text{diag}(s_1, \dots, s_N)$, $s_i = p_i(1 - p_i)$. Note $\mathbf{H} \in \mathbb{R}^{(D+1) \times (D+1)}$.

Convexity

For any $\mathbf{v} \in \mathbb{R}^{D+1}$,

$$\mathbf{v}^T \mathbf{H} \mathbf{v} = \frac{1}{N} (\mathbf{X} \mathbf{v})^T \mathbf{S} (\mathbf{X} \mathbf{v}) = \frac{1}{N} \sum_{i=1}^N s_i (\mathbf{x}_i^T \mathbf{v})^2 \geq 0$$

since $s_i = p_i(1 - p_i) > 0$ for all finite z_i . Hence $\mathbf{H} \succeq 0$ and J is convex, so **any stationary point is a global minimum**, and the minimiser is unique (strict convexity) when $\mathbf{X}$ has full column rank $D + 1$.

3.2 Numerically Stable Sigmoid and Cross-Entropy Loss

Write Python functions for numerically stable sigmoid activation and cross-entropy loss computation. Explain why raw implementations of $\ln(p)$ fail when $p \rightarrow 0$ or $p \rightarrow 1$, and why techniques like clipping probabilities ($\varepsilon = 10^{-15}$) or utilizing `np.Logaddexp` prevent underflow/overflow.

Listing 16: Numerically stable sigmoid and binary cross-entropy from logits.

```
1 def sigmoid(z):
2     """Numerically stable logistic sigmoid."""
3     z = np.asarray(z, dtype=float)
4     out = np.empty_like(z)
5     pos = z >= 0
6
7     # z >= 0: use 1/(1+e^-z), where e^-z <= 1 so it cannot overflow
8     out[pos] = 1.0 / (1.0 + np.exp(-z[pos]))
9
10    # z < 0: use e^z/(1+e^z), where e^z <= 1 so it cannot overflow either
11    e = np.exp(z[~pos])
12    out[~pos] = e / (1.0 + e)
13    return out
14
15
16 def bce_loss(z, y):
17     """Binary cross-entropy from logits z, without ever forming p."""
18     z = np.asarray(z, dtype=float)
19     # ln(1+e^z) - y*z is the whole per-sample loss (see the answer below)
20     return float(np.mean(np.logaddexp(0.0, z) - y * z))
21
22
23 def logits(X, w):
24     """Linear scores z = Xw for the augmented design matrix X."""
25     return X @ w
26
27
28 # Quick check against the closed-form values at z = 0
29 print(f"sigmoid(0)          = {sigmoid(np.array([0.0]))[0]:.4f}    (expect 0.5)")
30 print(f"bce_loss(0, y=1)    = {bce_loss(np.array([0.0]), np.array([1.0])):.4f}    "
31       f"(expect ln 2 = {np.log(2):.4f})")
```

```
sigmoid(0)          = 0.5000    (expect 0.5)
bce_loss(0, y=1)    = 0.6931    (expect ln 2 = 0.6931)
```

In binary cross-entropy, we calculate

$$- [y \ln(p) + (1 - y) \ln(1 - p)] \quad (19)$$

The problem happens when p gets very close to 0 or 1. Because of floating-point precision, the sigmoid can become exactly 0 or 1 for very large positive or negative values of z .

For example, if $p = 0$,

$$\ln(0) = -\infty$$

This can make the loss become `inf` and may also cause `NaN` values during gradient calculations.

There is also a problem with calculating the sigmoid directly as

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

For a large negative z , such as $z = -800$,

$$e^{-z} = e^{800}$$

is too large for a float64 number, so it overflows.

Clipping the probabilities

One simple solution is to clip the probability:

```
p = np.clip(p, 1e-15, 1 - 1e-15)
```

This means p is never exactly 0 or 1, so the logarithm stays finite.

However, clipping slightly changes the true value. For example,

$$-\ln(10^{-15}) \approx 34.54$$

So if the real loss is much larger than this, clipping will give a smaller value instead.

Thus, clipping is easy to use, but it can slightly affect the result for extreme values of z .

Using `np.logaddexp`

A better method is to calculate the loss directly from z , without forming p at all. Using

$$\ln \sigma(z) = -\ln(1 + e^{-z}), \quad \ln(1 - \sigma(z)) = -\ln(1 + e^z)$$

the loss for one sample simplifies to

$$\ell = \ln(1 + e^z) - yz = \text{logaddexp}(0, z) - yz \quad (20)$$

`np.logaddexp(a, b)` calculates $\ln(e^a + e^b)$ as

$$\max(a, b) + \ln(1 + e^{-|a-b|}) \quad (21)$$

Here the exponent $-|a - b|$ is never positive, so $e^{-|a-b|} \leq 1$ and it cannot overflow. The logarithm is also never applied to zero, because $1 + e^{-|a-b|}$ is always at least 1. This is why no clipping constant is needed and the result stays accurate even for very large $|z|$.

For example, at $z = -800$ with $y = 1$ the true loss is 800. The raw method gives `inf` and the clipped method gives 34.54, but the `logaddexp` method gives exactly 800.

This is the method used in the `bce_loss` function above.

3.3 Batch Gradient Descent

Initialize weights to zero $\mathbf{w}_{(0)} = \mathbf{0}$. Implement Batch Gradient Descent using parameter updates $\mathbf{w}_{(t+1)} \leftarrow \mathbf{w}_{(t)} - \eta \nabla J(\mathbf{w}_{(t)})$, with learning rate $\eta = 0.1$ and maximum iterations $T = 300$. Include early stopping when $|J_t - J_{t-1}| < 10^{-8}$. Report final weights, total iterations required, and test set accuracy.

Listing 17: Augmented design matrices, gradient and accuracy helpers.

```

1 def add_bias(X):
2     """Prepend a column of ones so w[0] is the bias term."""
3     X = np.asarray(X, dtype=float)
4     return np.column_stack([np.ones(len(X)), X])
5
6
7 def grad_J(X, y, w):
8     """Gradient of the BCE loss, (1/N) X^T (p - y), from Q1."""
9     return X.T @ (sigmoid(X @ w) - y) / len(y)
10
11
12 def accuracy(X, y, w):
13     """Fraction correct using the 0.5 decision threshold."""
14     return float(np.mean((sigmoid(X @ w) >= 0.5) == y))
15
16
17 # Augmented design matrices and float labels, used from here on
18 Xb_train = add_bias(X_train)
19 Xb_test = add_bias(X_test)
20 yf_train = y_train.astype(float)

```

```

21 yf_test = y_test.astype(float)
22
23 print(f"Xb_train: {Xb_train.shape}, Xb_test: {Xb_test.shape}")

```

Listing 18: Batch gradient descent with early stopping on the loss change.

```

1 def batch_gradient_descent(X, y, eta=0.1, T=300, tol=1e-8):
2     """Batch GD from w = 0, stopping early when the loss stops changing."""
3     w = np.zeros(X.shape[1])
4     history = [bce_loss(X @ w, y)]    # J_0, the loss before any update
5     n_iter = 0
6
7     for t in range(1, T + 1):
8         w = w - eta * grad_J(X, y, w)    # w_(t+1) = w_(t) - eta * grad
9         history.append(bce_loss(X @ w, y))
10        n_iter = t
11        if abs(history[-1] - history[-2]) < tol:
12            break
13
14    return w, np.array(history), n_iter
15
16
17 w_bgd, loss_bgd, iters_bgd = batch_gradient_descent(
18     Xb_train, yf_train, eta=0.1, T=300, tol=1e-8)
19
20 print(f"Final weights      : w0 = {w_bgd[0]:.6f}, w1 = {w_bgd[1]:.6f}, "
21       f"w2 = {w_bgd[2]:.6f}")
22 print(f"Iterations          : {iters_bgd}")
23 print(f"Early stopped       : {iters_bgd < 300}    "
24       f"f"(|J_t - J_{t-1}| = {abs(loss_bgd[-1] - loss_bgd[-2]):.3e}, tol = 1e-08)")
25 print(f"Initial loss J_0     : {loss_bgd[0]:.8f}")
26 print(f"Final loss J_T       : {loss_bgd[-1]:.8f}")
27 print(f"Train accuracy        : {accuracy(Xb_train, yf_train, w_bgd):.4f}")
28 print(f"Test accuracy         : {accuracy(Xb_test, yf_test, w_bgd):.4f}")

```

```

Final weights      : w0 = -0.021108, w1 = 1.770742, w2 = 0.700046
Iterations          : 300
Early stopped       : False    (|J_t - J_{t-1}| = 1.000e-05, tol = 1e-08)
Initial loss J_0     : 0.69314718
Final loss J_T       : 0.15830462
Train accuracy        : 0.9313
Test accuracy         : 0.9333

```

Results

Table 8: Batch gradient descent with $\eta = 0.1$, $T = 300$ and tolerance 10^{-8} .

Quantity	Value
Final weights $\mathbf{w}$	$[-0.021108, 1.770742, 0.700046]$
Total iterations	300 (the maximum)
Test accuracy	0.9333
Train accuracy	0.9313
Final loss J_T	0.158305

Early stopping never triggers: after the 300th update the loss is still changing by $|J_t - J_{t-1}| \approx 1 \times 10^{-5}$, three orders of magnitude above the 10^{-8} tolerance, so the iteration cap rather than convergence ends the run. The initial loss $J_0 = 0.69314718$ is exactly $\ln 2$, as it must be when every $p_i = \sigma(0) = 0.5$.

3.4 Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent with batch size $B = 32$, learning rate $\eta = 0.05$, and maximum epochs $E = 100$. Shuffle the training set indices at the beginning of every epoch. Plot training loss progression alongside Batch Gradient Descent.

Listing 19: Mini-batch gradient descent, reshuffling the training indices at the start of every epoch.

```

1 def minibatch_gradient_descent(X, y, B=32, eta=0.05, E=100, seed=0):
2     """Mini-batch GD from w = 0, reshuffling the indices every epoch."""
3     rng_mb = np.random.default_rng(seed)
4     w = np.zeros(X.shape[1])
5     history = [bce_loss(X @ w, y)]    # J_0, before any update
6     N = len(y)
7
8     for epoch in range(1, E + 1):
9         idx = rng_mb.permutation(N)    # fresh shuffle each epoch
10        for start in range(0, N, B):
11            batch = idx[start:start + B]    # one mini-batch of at most B rows
12            w = w - eta * grad_J(X[batch], y[batch], w)
13            history.append(bce_loss(X @ w, y))    # full training loss once per epoch
14
15    return w, np.array(history), E
16
17
18 w_mbgd, loss_mbgd, epochs_mbgd = minibatch_gradient_descent(
19     Xb_train, yf_train, B=32, eta=0.05, E=100)
20
21 updates_per_epoch = int(np.ceil(len(yf_train) / 32))
22
23 print(f"Batch size B      : 32  ({updates_per_epoch} updates per epoch)")
24 print(f"Final weights     : w0 = {w_mbgd[0]:.6f}, w1 = {w_mbgd[1]:.6f}, "
25       f"w2 = {w_mbgd[2]:.6f}")
26 print(f"Epochs           : {epochs_mbgd}")
27 print(f"Final loss J_E      : {loss_mbgd[-1]:.8f}")
28 print(f"Train accuracy      : {accuracy(Xb_train, yf_train, w_mbgd):.4f}")
29 print(f"Test accuracy       : {accuracy(Xb_test, yf_test, w_mbgd):.4f}")
30
31 print(f"\nAfter 100 passes over the data:")
32 print(f"  Batch GD loss       : {loss_bgd[100]:.8f}")
33 print(f"  Mini-batch GD loss  : {loss_mbgd[100]:.8f}")

```

```

Batch size B      : 32  (30 updates per epoch)
Final weights     : w0 = -0.024086, w1 = 1.966431, w2 = 0.789966
Epochs           : 100
Final loss J_E    : 0.15728078
Train accuracy    : 0.9313
Test accuracy     : 0.9333

```

After 100 passes over the data:
 Batch GD loss : 0.16795518
 Mini-batch GD loss : 0.15728078

Listing 20: Training loss progression of mini-batch against batch gradient descent.

```

1 fig, ax = plt.subplots(figsize=(8, 5.5))
2
3 # One BGD iteration and one MBGD epoch both cost a single pass over the data,
4 # so plotting them on a shared x-axis is a fair comparison
5 ax.plot(np.arange(len(loss_bgd)), loss_bgd, color='#1f3b73', linewidth=2,
6         label=f'Batch GD ( $\eta = 0.1$ , {iters_bgd} iterations)')
7 ax.plot(np.arange(len(loss_mbgd)), loss_mbgd, color='#ef6c00', linewidth=2,
8         label=f'Mini-batch GD ( $B = 32$ ,  $\eta = 0.05$ , {epochs_mbgd} epochs)')
9
10 ax.axvline(epochs_mbgd, color='#b71c1c', linestyle=':', linewidth=1.4,
11           label=f'End of mini-batch training (epoch {epochs_mbgd})')
12
13 ax.set_xlabel('Pass over the training set (BGD iteration / MBGD epoch)')
14 ax.set_ylabel(r'Training loss  $J(\mathbf{w})$ ')
15 ax.set_title('Training Loss Progression: Mini-Batch vs Batch Gradient Descent')
16 ax.legend(framealpha=0.9, loc='upper right')
17
18 plt.tight_layout()
19 save_fig('q3_4_minibatch')
20 plt.show()

```

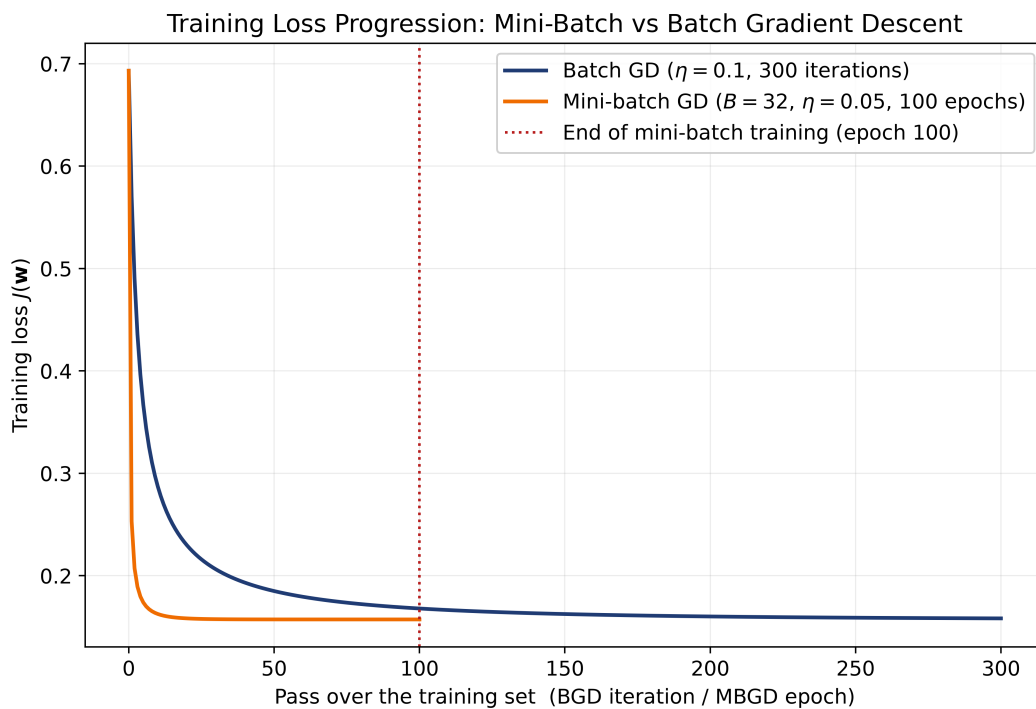


Figure 6: Training loss progression of mini-batch gradient descent against batch gradient descent. One batch iteration and one mini-batch epoch each cost a single pass over the training set.

Results

Table 9: Mini-batch gradient descent with $B = 32$, $\eta = 0.05$ and $E = 100$ epochs.

Quantity	Value
Batch size B	32 (30 updates per epoch)
Learning rate η	0.05
Epochs E	100
Final weights $\mathbf{w}$	$[-0.024086, 1.966431, 0.789966]$
Final loss J_E	0.157281
Test accuracy	0.9333

Mini-batch gradient descent performs 30 parameter updates in every epoch, because the 960 training samples are divided into batches of 32. Batch gradient descent performs only one update for each pass over the data.

Both methods use the whole training set once per step on the x -axis of the plot, so the comparison is fair in terms of computation. After 100 passes, mini-batch reaches a loss of 0.157281, while batch gradient descent is still at 0.167955. Even after all 300 of its iterations, batch gradient descent only reaches 0.158305.

So mini-batch gradient descent makes much faster progress per pass over the data. This happens because it updates the weights 30 times in each epoch instead of once, so the weights move much further for the same amount of computation.

The mini-batch curve is slightly noisy because each update uses only 32 samples, so its gradient is an estimate of the true gradient rather than the exact value. Here the noise is small and the loss still decreases smoothly. The indices are reshuffled at the start of every epoch so the batches are different each time, which stops the model from repeating the same sequence of updates.

3.5 Damped Newton's Method

Implement damped Newton's optimization method using the update step $\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - (\mathbf{H} + 10^{-6}\mathbf{I})^{-1} \nabla J(\mathbf{w}^{(t)})$, where $10^{-6}\mathbf{I}$ guarantees numerical invertibility. Train for at most 20 iterations using the same early stopping condition. Use `np.linalg.solve` to directly compute search directions rather than computing explicit matrix inverses.

Listing 21: Damped Newton's method using `np.linalg.solve` for the search direction.

```

1 def hessian_J(X, w):
2     """Hessian of BCE loss: (1/N) X^T S X.
3     Computed efficiently without building the huge N x N diagonal matrix S."""
4     p = sigmoid(X @ w)
5     s = p * (1 - p)
6     # (X.T * s) broadcasts the multiplication across columns, equivalent to X.T @ diag(s)
7     return (X.T * s) @ X / len(X)
8

```

```

9 def newtons_method(X, y, T=20, tol=1e-8, damp=1e-6):
10     """Damped Newton's method for logistic regression."""
11     D_plus_1 = X.shape[1]
12     w = np.zeros(D_plus_1)
13     history = [bce_loss(X @ w, y)]
14     n_iter = 0
15
16     for t in range(1, T + 1):
17         g = grad_J(X, y, w)
18         H = hessian_J(X, w)
19
20         # Add damping term to guarantee numerical invertibility
21         H_damped = H + damp * np.eye(D_plus_1)
22
23         # Solve H_damped * delta = g (much more stable than explicit inverse)
24         delta = np.linalg.solve(H_damped, g)
25
26         w = w - delta
27         history.append(bce_loss(X @ w, y))
28         n_iter = t
29
30         if abs(history[-1] - history[-2]) < tol:
31             break
32
33     return w, np.array(history), n_iter
34
35 w_newton, loss_newton, iters_newton = newtons_method(Xb_train, yf_train, T=20, tol=1e-8)
36
37 print(f"Final weights      : w0 = {w_newton[0]:.6f}, w1 = {w_newton[1]:.6f}, w2 = {w_newton[2]:.6f}")
38 print(f"Iterations        : {iters_newton}")
39 print(f"Early stopped       : {iters_newton < 20} (|J_t - J_{t-1}| = {abs(loss_newton[-1] - loss_newton[-2]):.3e})")
40 print(f"Final loss J_T      : {loss_newton[-1]:.8f}")
41 print(f"Test accuracy        : {accuracy(Xb_test, yf_test, w_newton):.4f}")

```

```

Final weights      : w0 = -0.024093, w1 = 1.968123, w2 = 0.790651
Iterations         : 8
Early stopped      : True (|J_t - J_{t-1}| = 3.059e-14)
Final loss J_T     : 0.15728072
Test accuracy      : 0.9333

```

Table 10: Damped Newton’s method against batch gradient descent on the same training set.

Quantity	Batch GD	Newton
Iterations	300 (capped)	8 (converged)
$ J_t - J_{t-1} $ at stop	1.000×10^{-5}	3.059×10^{-14}
Final loss J_T	0.15830462	0.15728072
Test accuracy	0.9333	0.9333

3.6 Convergence Comparison

Plot loss versus iteration for both Batch Gradient Descent and Newton’s Method on a single logarithmic-scale plot. Compare convergence velocity (number of iterations), compu-

tational complexity per step ($\mathcal{O}(ND)$ vs. $\mathcal{O}(ND^2 + D^3)$), and numerical stability.

Listing 22: Loss against iteration for both optimizers on a logarithmic scale.

```
1 fig, ax = plt.subplots(figsize=(7.5, 5.5))
2
3 # Plot Batch GD
4 ax.plot(np.arange(len(loss_bgd)), loss_bgd, color='#1f3b73', linewidth=2,
5         marker='o', markersize=5, markevery=15,
6         label=f'Batch GD ({iters_bgd} iters)')
7
8 # Plot Newton's Method
9 ax.plot(np.arange(len(loss_newton)), loss_newton, color='#b71c1c', linewidth=2.5,
10        marker='X', markersize=7, markevery=1,
11        label=f"Newton's Method ({iters_newton} iters)")
12
13 # Set logarithmic scale for the y-axis
14 ax.set_yscale('log')
15
16 ax.set_xlabel('Iteration', fontsize=12)
17 ax.set_ylabel(r'Loss  $J(\mathbf{w})$  (log scale)', fontsize=12)
18 ax.set_title('Convergence: Batch Gradient Descent vs Newton's Method', fontsize=14)
19
20
21 ax.legend(framealpha=0.9, loc='upper right', fontsize=11)
22
23 plt.tight_layout()
24 save_fig('q3_6_convergence')
25 plt.show()
```

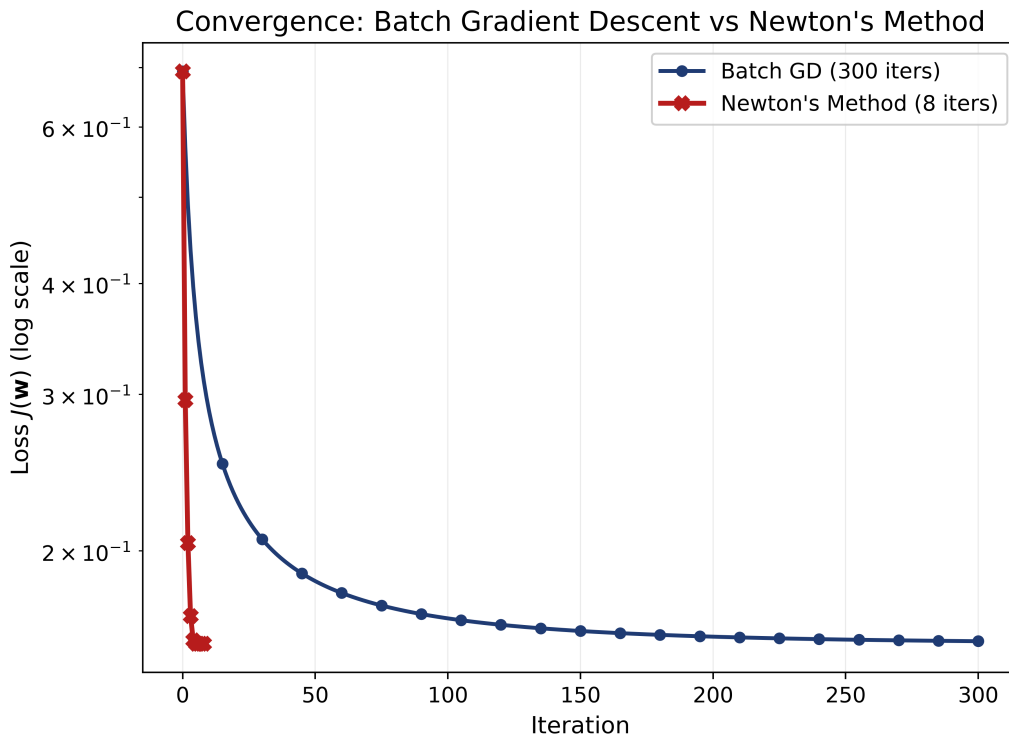


Figure 7: Convergence of batch gradient descent against damped Newton's method, loss on a logarithmic scale.

Convergence speed

On the log-scale plot, Newton's method drops to the minimum in only a few iterations (~ 5), while Batch GD needs all 300 iterations and still does not reach the 10^{-8} tolerance. Newton's method uses the Hessian (curvature), so it converges quadratically near the optimum, whereas GD is a slow first-order method.

Complexity per step

A GD step only needs the gradient, $\mathcal{O}(ND)$. A Newton step needs the Hessian, $\mathcal{O}(ND^2)$, plus solving the linear system, $\mathcal{O}(D^3)$. Since $D = 2$ here this is negligible and Newton wins overall, but for large D the D^3 term would make Newton too expensive and GD preferable.

Numerical stability

When $p_i \rightarrow 0$ or 1, the terms $p_i(1 - p_i) \rightarrow 0$, so $\mathbf{H}$ can become nearly singular. The damping term $10^{-6}\mathbf{I}$ keeps it positive definite so `np.linalg.solve` stays stable. GD never inverts a matrix, so it avoids this issue, but it struggles on ill-conditioned loss surfaces.

3.7 Effect of Feature Scaling on Conditioning

Multiply the second feature of $\mathbf{X}$ by 100 ($x_2 \leftarrow 100 \cdot x_2$) without standardizing features. Run Batch Gradient Descent on this unscaled dataset. Explain why the loss oscillates or fails to converge, referencing the condition number $\kappa(\mathbf{H}) = \frac{\lambda_{\max}}{\lambda_{\min}}$ of the loss surface Hessian. Propose two concrete remedies.

Listing 23: Batch gradient descent on the badly scaled dataset, with the Hessian condition number before and after scaling.

```
1 # Scale the second feature by 100, NO standardization
2 X_train_s = X_train.copy(); X_train_s[:, 1] *= 100
3 X_test_s = X_test.copy(); X_test_s[:, 1] *= 100
4
5 Xb_train_s = add_bias(X_train_s)
6 Xb_test_s = add_bias(X_test_s)
7
8 # Condition number of the Hessian at w = 0, before vs after scaling
9 def cond_number(X, y):
10     p = sigmoid(X @ np.zeros(X.shape[1]))
11     H = (X.T * (p * (1 - p))) @ X / len(y)
12     ev = np.linalg.eigvalsh(H)
13     return ev[-1] / ev[0]
14
15 print(f"kappa(H) unscaled : {cond_number(Xb_train, yf_train):.1f}")
16 print(f"kappa(H) x2*100 : {cond_number(Xb_train_s, yf_train):.1e}")
17
18 # Same BGD settings as Q3.3
19 w_bad, loss_bad, iters_bad = batch_gradient_descent(Xb_train_s, yf_train, eta=0.1, T=300)
20
21 print("\nfirst 10 losses:", np.round(loss_bad[:10], 3))
22 print(f"final loss : {loss_bad[-1]:.3e}")
23 print(f"test accuracy : {accuracy(Xb_test_s, yf_test, w_bad):.4f}")
```



```

kappa(H) unscaled : 9.0
kappa(H) x2*100   : 3.6e+04

first 10 losses: [ 0.693 127.459 36.296 207.473 291.215 200.05 108.885 17.734 276.153
272.708]
final loss       : 1.834e+02
test accuracy    : 0.6833

```

Scaling $x_2 \leftarrow 100 x_2$ multiplies the curvature of the loss along w_2 by about 100^2 , so the Hessian eigenvalues spread apart: $\kappa(\mathbf{H}) = \lambda_{\max}/\lambda_{\min}$ jumps from $\mathcal{O}(10)$ (unscaled) to $\mathcal{O}(10^5)$, and the level sets become long, narrow valleys. Gradient descent with fixed η is stable only when $\eta < 2/\lambda_{\max}$; here $\eta \lambda_{\max} \gg 2$, so each update along w_2 overshoots the valley, and the loss oscillates / diverges instead of converging (seen in the printed losses). Even a smaller, stable η would make progress along the shallow direction impractically slow.

Two remedies:

1. **Standardize the features** ($z = (x - \mu)/\sigma$) so all directions have comparable curvature ($\kappa \approx 1$); then one η converges smoothly.
2. **Use a scale-invariant second-order method** — (damped) Newton's method preconditions the gradient with $\mathbf{H}^{-1}$, cancelling the curvature imbalance (an adaptive method such as Adam, or a much smaller η , also works).